

PROOF-CARRYING STREAMED WORKFLOW GRAPHS

P. KINGSTON¹, T. FAGNONI²

^{1,2}*AppliedAI, Abu Dhabi, United Arab Emirates*

¹phillip.kingston@opus.com, ²theo.fagnoni@opus.com

Incremental construction of directed workflow graphs can be formulated as a constrained graph-extension problem: each proposed addition should preserve the existence of a viable completion. In AI systems, an emitted addition can be syntactically correct even when its evidence is inadequate, its scope is invalid, or it makes the remaining policy obligations unsatisfiable. We introduce *proof-carrying streams*, in which every addition updates the committed graph and transforms a certificate witnessing that at least one acceptable completion remains.

We model each step in the graph construction by a state s , which records the current committed graph and all information needed to apply future additions and determine whether the construction remains viable. For an arbitrary state space S , an alphabet Σ of possible additions, a deterministic transition function $\delta : S \times \Sigma \rightarrow S$ that assigns a unique successor state to each state and addition, and a set F of acceptable states, we define the canonical *viability residual* $\mathcal{R}_s$. The viability residual $\mathcal{R}_s$ contains exactly the finite addition streams that can be admitted from s while leaving the resulting state viable, i.e., still able to reach an accepting state by some finite continuation. As additions are admitted, the viability residual evolves to describe exactly the futures that remain viable from the resulting state. A certificate is sound at a construction state when every future stream it represents belongs to that state's viability residual. Graph construction that begins with a non-empty sound certificate and proceeds through sound certificate updates preserves completion viability through every admitted addition.

We encode each construction state into a proof state that retains enough information to determine completion viability. Without loss of generality, we restrict every proof state to the image of this encoding, so every proof state represents at least one construction state. A proof-state abstraction consists of the encoding, an update rule that computes the next proof state from the current proof state and the next addition, and a classification of proof states as viable or nonviable. The abstraction is viability-exact when two conditions hold: updating an encoded proof state produces the same result as first applying the addition to the construction state and then encoding its successor, and the proof state's viability classification agrees exactly with the viability of every construction state it represents. These requirements imply that construction states encoded as the same proof state must have equal viability residuals. Consequently, grouping construction states by equality of their residuals forms the coarsest viability-exact abstraction: every viability-exact abstraction must keep construction states with different residuals separate. A finite viability-exact abstraction exists if and only if the construction states have finitely many distinct residuals.

Keywords: proof-carrying streams, workflow graphs, viability residual, residual languages, Nerode equivalence, incremental verification.

2020 Mathematics Subject Classification: 68Q45, 68Q70, 68Q60, 68T27